Rcpp Style Guide

Ivan Jacob Agaloos Pesigan

This guide extends the R style guide to C++. Where it is silent, the R guide applies, and then the Google C++ style guide (https://google.github.io/styleguide/cppguide.html). C++ keeps its `return`. Agents follow `.setup/ai/style/rcpp.md`, which follows this guide and is changed together with it.

1 Files

- Sources are in `.setup/cpp`, with `Makevars` and `Makevars.win`. `Build()` copies them to `src`; never edit `src` directly.
- Every project with C++ has `.setup/cpp/000-forward-declarations.cpp`, which declares every function, so the order of the other files does not matter. It is the only numbered file.
- One function per file: `<pkg>-<name>.cpp` for a function exported to users, `<pkg>-<name>-dot.cpp` for an internal one. Numbered helper files (`001-transformation-helpers.cpp`) are split into these.
- Each file starts with a header block whose `edit` line names the file itself.

```
// ----------------------------------------------------------------------------
// edit .setup/cpp/template-foo-dot.cpp
// Ivan Jacob Agaloos Pesigan
// ----------------------------------------------------------------------------
```

2 Types

- RcppArmadillo by default (`arma::mat`, `arma::vec`), with `// [[Rcpp::depends(RcppArmadillo)]]`.
- Plain Rcpp types only at the R boundary (`Rcpp::List`, `Rcpp::Named`).
- Matrices and vectors are `const` references unless the function changes them; scalars are passed by value.

3 Names

- Functions in `BigCamelCase`; arguments and variables in `snake_case`.
- Explicit namespaces: `arma::`, `Rcpp::`, `std::`. No `using namespace`.
- Every function has complete roxygen comments (`//'`), as in the R style guide.
- Internal functions are exported to R with a dot: `// [[Rcpp::export(.Foo)]]`, with `@noRd`. Public functions have `// [[Rcpp::export]]`.
- A helper library's C++ follows these rules, in `.setup/cpp/`. A project never edits or restyles a copy; it is changed only in its library.

4 Bodies

- Every `if`, `else`, `for`, and `while` body is in braces. `make style` inserts them.
- Lines under 80 characters, including roxygen comments.
- Sanity checks go in the R wrapper (`.CheckFoo()`). `Rcpp::stop()` in C++ is only for numerical failures.
- `// # nocov start` and `// # nocov end` around unreachable branches only.

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::export(.Scale)]]
arma::mat Scale(const arma::mat& x_mat, const double scale) {
  if (scale == 0.0) {
    Rcpp::stop("scale is zero.");
  }
  return x_mat / scale;
}
```

5 Efficiency

Packages maximize efficiency.

- Move a function to C++ when a measurement shows it is worth it, above all what simulations and bootstraps call many times.
- A faster version wins, even if it is harder to read. This guide's rules still apply in full.
- Tests guard correctness: they pass on the R version first, and the C++ version must pass the same tests.
- Put timings in `.setup/tests-benchmark/`.

6 Tests

C++ functions are tested from R, as R functions, in `tests/testthat`.

7 Tools

- `make style`: `clang-format` (Google style, `InsertBraces`).

- `make lint`: the header line, forward declarations, file names, names, `using namespace`, `const` references, and the format. Warnings in rProject 0.0.63; errors from 0.0.64.

References

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/